IT Department
Networks & Cybersecurity

Python Codes of Automation

Code 1

```
import getpass
import telnetlib

HOST = "192.168.200.254"
user = input("Username: ")
password = getpass.getpass()

tn = telnetlib.Telnet(HOST)

tn.read_until(b"Username: ")
tn.write(user.encode('ascii') + b"\n")
if password:
    tn.read_until(b"Password: ")
    tn.write(password.encode('ascii') + b"\n")

tn.write(b"enable\n")
tn.write(b"we\n")
tn.write(b"conf t\n")
tn.write(b"interface loopback0\n")
tn.write(b"ip address 1.1.1.1 255.255.255.255\n")
tn.write(b"end\n")
tn.write(b"exit\n")

print(tn.read_all().decode('ascii'))
```

Code 2

```
from netmiko import ConnectHandler
# Define device details
R1 = {
    "device_type": "cisco_ios",
    "host": "192.168.200.254",
    "username": "we",
    "password": "we",
    "secret": "we",
}
try:
    net_connect = ConnectHandler(**R1)
net_connect.enable()
    print(net_connect.send_command("show ip interface
brief"))
    net_connect.disconnect() except
Exception as e:     print(f"Error:
{e}")
```

Code 3

```
import telnetlib host =
"192.168.200.253" user = "we" password
= "we" tn = telnetlib.Telnet(host)
tn.read_until(b"Username: ")
tn.write(user.encode('ascii') + b"\n")
if password:
    tn.read_until(b"Password: ")
tn.write(password.encode('ascii') + b"\n")
tn.write(b"enable\n") tn.write(b"we\n")
tn.write(b"conf t\n") tn.write(b"vlan 2\n")
tn.write(b"exit\n") tn.write(b"vlan 3\n")
tn.write(b"exit\n") tn.write(b"vlan 4\n")
tn.write(b"exit\n") tn.write(b"end\n")
tn.write(b"exit\n")
print(tn.read_all().decode('ascii'))
```

Code 4

```
from netmiko import ConnectHandler
# Define device details devices
= [
    {
        "device_type": "cisco_ios",     # IP address of R1
        "host": "192.168.200.254",     # Username for SSH
        "username": "we",              # Password for SSH
        "password": "we",              # Enable secret
password
        "secret": "we",
```

```
    },
    {
        "device_type": "cisco_ios",    # IP address of IOU1
        "host": "192.168.200.253",     # Username for SSH
        "username": "we",
        "password": "we",
        "secret": "we"
    }
]
# Loop through devices and save configurations
for device in devices:     try:
        # Establish SSH connection
net_connect = ConnectHandler(**device)
net_connect.enable()

        # Set terminal length to 0 to disable pagination
net_connect.send_command("terminal length 0")
        # Get the hostname of the device
        hostname = net_connect.find_prompt().strip("#")
        # Pull running configuration
       config = net_connect.send_command("show running-
config")
        # Save configuration to a file named after the
hostname
        filename = f"config_{hostname}.txt"
with open(filename, "w") as file:
            file.write(config)
        print(f"Configuration saved for {hostname} in
{filename}")
        # Disconnect the session
net_connect.disconnect()     except Exception as e:
print(f"Failed to retrieve configuration from
{device['host']}: {e}")
```

Code 5

```
from netmiko import ConnectHandler
from ipaddress import ip_network
# Define device details devices
= [
    {
        "device_type": "cisco_ios",
        "host": "192.168.200.254",  # R1
        "username": "we",
        "password": "we",
        "secret": "we",  # Enable secret
    },
    {
        "device_type": "cisco_ios",
        "host": "192.168.200.252",  # R2
        "username": "we",
        "password": "we",
        "secret": "we",
    }
]
```

```
# OSPF Configuration
ospf_process_id = 1  # OSPF process ID
for device in devices:     try:
        # Connect to the device
        net_connect = ConnectHandler(**device)
net_connect.enable()
        # Get the list of interfaces and their subnet masks
output = net_connect.send_command("show ip interface brief")
        interfaces = output.splitlines()
        # List to hold OSPF configuration commands
ospf_commands = [f"router ospf {ospf_process_id}"]
for line in interfaces[1:]:  # Skip the header line
if "up" in line and "up" in line.split()[-1]:  #
Check for active interfaces
parts = line.split()
ip_address = parts[1]                    if
ip_address != "unassigned":                  #
Retrieve the subnet mask using "show run
interface <interface>"
                    interface_name = parts[0]
int_output =
net_connect.send_command(f"show run interface
{interface_name}")
                    for config_line in
int_output.splitlines():                          if "ip
address" in config_line:
                            _, ip, subnet_mask =
config_line.split()[-3:]
                            # Calculate the network and
wildcard mask
                            network =
ip_network(f"{ip}/{subnet_mask}", strict=False)
ospf_commands.append(
f"network
```

```
{network.network_address} {network.hostmask} area 0"
                                )
        # Send OSPF configuration commands to the device
net_connect.send_config_set(ospf_commands)
print(f"OSPF configured on {device['host']}:")
for cmd in ospf_commands:
            print(cmd)
        # Save configuration
net_connect.save_config()
net_connect.disconnect()     except
Exception as e:          print(f"Failed to
configure OSPF on
{device['host']}: {e}")
```

Code 6

```
 import getpass
import
telnetlib
HOST = "192.168.126.201" user =
input("Username: ") password =
getpass.getpass() tn =
telnetlib.Telnet(HOST)
tn.read_until(b"Username: ")
tn.write(user.encode('ascii') + b"\n")
if password:
 tn.read_until(b"Password: ")
tn.write(password.encode('ascii') + b"\n")
tn.write(b"enable\n") tn.write(b"we\n")
tn.write(b"conf t\n")
for vlan_id in range(2, 12):  # This will create VLANs from
2 to 11
    tn.write(f"vlan {vlan_id} \n".encode('ascii'))  # Create
VLAN
    tn.write(f"name WE-{vlan_id} \n".encode('ascii'))  # Set
VLAN name tn.write(b"end\n")
tn.write(b"exit\n")
print(tn.read_all().decode('ascii'))
```